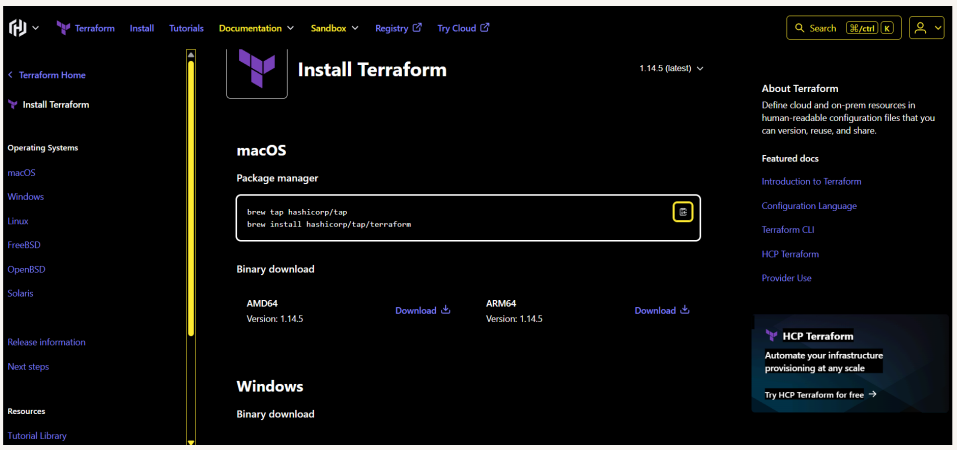
nextwork.org

Create S3 Buckets with Terraform

Michael K.



Michael K.
NextWork Student

Introducing Today's Project!

In this project, I will Install and configure Terraform, Configure your AWS credentials in the terminal, Create and manage S3 buckets with Terraform and Upload files to S3 using Terraform.

Tools and concepts

Services I used were IAM and S3. Key concepts I learnt include infrastructure as code, terraform, blocks, resource block, AWS CLI, provider block, terraform init, plan and apply.

Project reflection

This project took me approximately 1 hour.

I chose to do this project today because i wanted practical introduction into Terraform.

Michael K.
NextWork Student

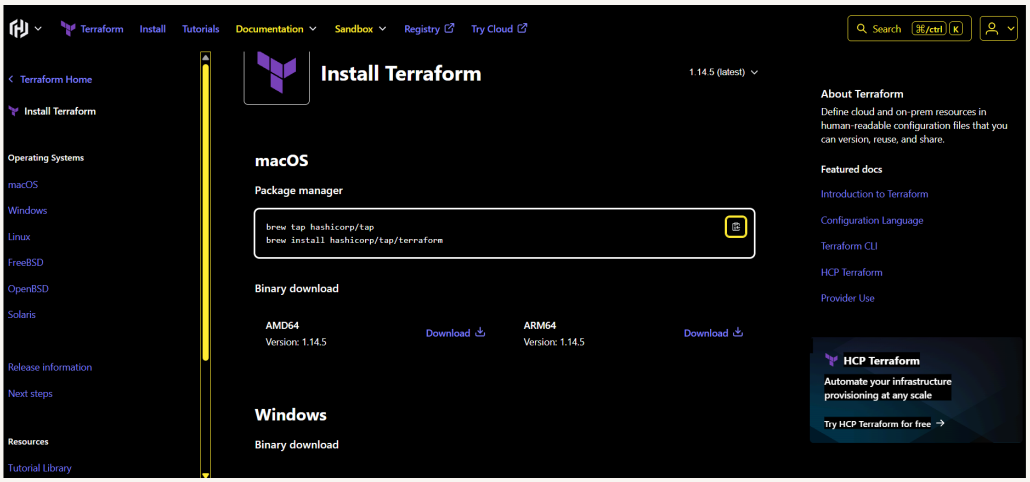
Introducing Terraform

Terraform is a tool that helps to build and manage cloud infrastructures using code. Instead of setting up resources manually in the AWS console or CLI, a script is written that tells Terraform exactly what the developer wants in his cloud infrastructure, like servers, databases, and networks. Terraform then automatically builds all of this for him, using his script as a blueprint.

Terraform is one of the most popular tools used for infrastructure as code (IaC), which is the practice of describing a cloud setup (like the servers, storage, networks) in plain text files instead of clicking through a web console. When these files are run, the cloud builds everything exactly as written. Because the description lives in code, it can be version-controled, shared with teammates, and the same environment any time recreated.

Terraform uses configuration files to define and manage infrastructure. These files describe the desired state of an infrastructure, and Terraform figures out how to achieve that state. main.tf is a central file in a Terraform project. It's where the infrastructure description is written down using Terraform's language. It is like the blueprint for building cloud resources.

Michael K.
NextWork Student



Michael K.
NextWork Student

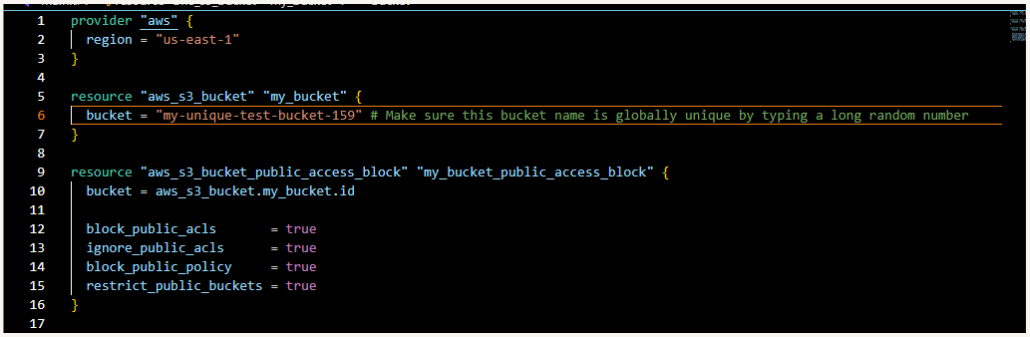
Configuration files

The configuration is structured in “blocks.” There are different kinds of blocks to do different kinds of things: 1. A provider block tells Terraform which cloud to work with e.g. AWS, Google Cloud, Azure. (this is in our `main.tf code!) 2. A resource block describes what to create, such as a storage bucket or virtual machine, and how it should look. By keeping each piece of the infrastructure in its own block, the file stays easy to read and one part can be modified without touching the rest of your setup!

My main.tf configuration has three blocks

The first block tells Terraform to use AWS. A "provider" is a plugin that lets Terraform work with a specific cloud service. The provider turns your configuration into API calls that create and manage your infrastructure. The second block creates an S3 bucket. my_bucket is an internal name that other blocks of code in main.tf will use to reference the bucket. The third block controls who can access the S3 bucket. All the settings like block_public_acls, ignore_public_acls, block_public_policy, and restrict_public_buckets are set to true to prevent public access to the bucket and its contents.

Michael K.
NextWork Student



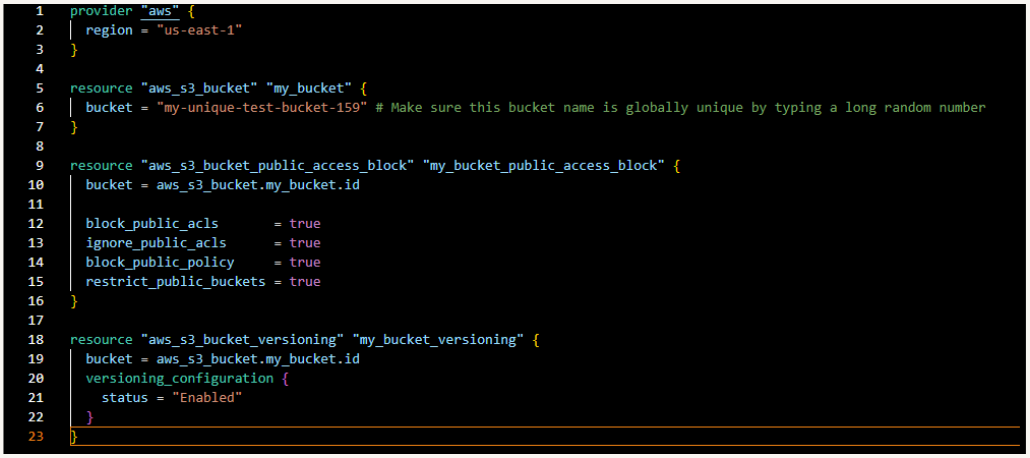
```
provider "aws" {
  region = "us-east-1"
}

resource "aws_s3_bucket" "my_bucket" {
  bucket = "my-unique-test-bucket-159" # Make sure this bucket name is globally unique by typing a long random number
}

resource "aws_s3_bucket_public_access_block" "my_bucket_public_access_block" {
  bucket = aws_s3_bucket.my_bucket.id

  block_public_acls       = true
  ignore_public_acls      = true
  block_public_policy     = true
  restrict_public_buckets = true
}

```

Michael K.
NextWork Student

Customizing my S3 Bucket

For my project extension, I visited the official Terraform documentation to find and read documentation about Terraform modules and providers. In my case, i've visited the registry to read documentation on the aws provider, where i found setup instructions, descriptions of each available resource, best practice tips and examples, and parameters for customization.

I chose to customise my bucket by enabling versioning.



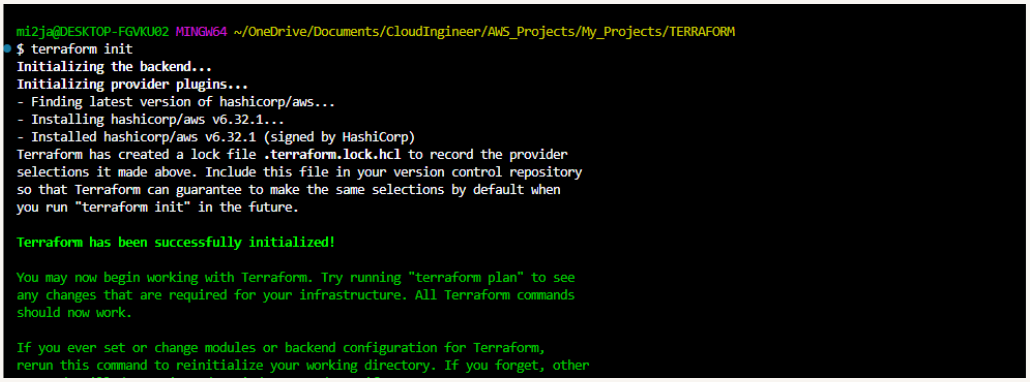
```
provider "aws" {
  region = "us-east-1"
}

resource "aws_s3_bucket" "my_bucket" {
  bucket = "my-unique-test-bucket-159" # Make sure this bucket name is globally unique by typing a long random number
}

resource "aws_s3_bucket_public_access_block" "my_bucket_public_access_block" {
  bucket = aws_s3_bucket.my_bucket.id

  block_public_acls       = true
  ignore_public_acls      = true
  block_public_policy     = true
  restrict_public_buckets = true
}

resource "aws_s3_bucket_versioning" "my_bucket_versioning" {
  bucket = aws_s3_bucket.my_bucket.id
  versioning_configuration {
    status = "Enabled"
  }
}
```

Michael K.
NextWork Student

Terraform commands

I ran 'terraform init' to sets up my project by: 1. Downloading necessary plugins: It grabs the providers which handle the connection with cloud services like AWS. 2. Setting up the backend: Terraform needs to set up a backend that keep its own record of my infrastructure, so future commands can see what has already been created. 3. Preparing modules: If my setup uses reusable code blocks (called modules) this is where the code is downloaded from their source to replace placeholders in my code. 4. Creating a lock file: This file keeps track of the versions of everything Terraform is using, making sure that everyone on my team is using the same setup to avoid inconsistencies. This detailed output from terraform init tells you about each step Terraform is taking to get ready, like finding and installing the right AWS provider.

Next, I ran 'terraform plan' to creates an execution plan, showing what changes Terraform will make to the infrastructure based on my configuration files (like main.tf). It shows what will be created, updated, or destroyed, so i can review and confirm the configuration before any real changes are made.


mi2ja@DESKTOP-FGVKU02 MINGW64 ~/OneDrive/Documents/CloudIngineer/AWS_Projects/My_Projects/TERRAFORM
$ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.32.1...
- Installed hashicorp/aws v6.32.1 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.
Terraform has been successfully initialized!
You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.
If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other

Michael K.
NextWork Student

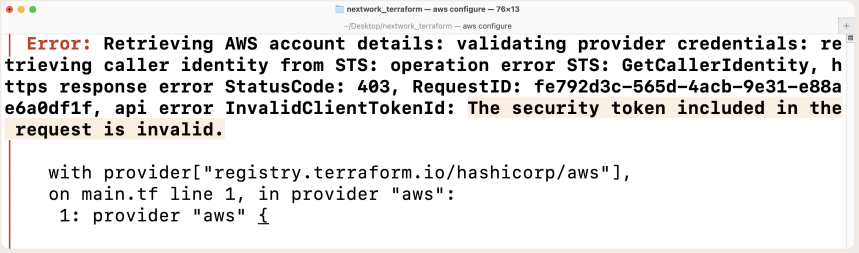
AWS CLI and Access Keys

When I tried to plan my Terraform configuration, I received an error message that says no credentias because Terraform doesn't have the necessary credentials to access my AWS account. i'll need to configure AWS credentials for Terraform to use, so that it can create AWS resources on my behalf.

To resolve my error, first I installed AWS CLI, which is a powerful tool that lets me manage my AWS services from my terminal. Instead of having to use the AWS Management Console, i can now run text commands from my local machine.

I set up AWS access keys to enable AWS CLI way to log into my management console throught my local CLI. The CLI and the Management Console aren't connected, so the CLI won't know if I've already authenticated yourself by logging into the console. It needs its own way to authenticate me, and accepting an access key is one of the fastest ways to do it.

Michael K.
NextWork Student



Michael K.
NextWork Student

Lanching the S3 Bucket

I ran 'terraform apply' to apply the changes i've written in my Terraform configuration. It creates, modifies, or deletes resources in my infrastructure based on my code. In my case, this command creates the S3 bucket in your AWS account.

The sequence of running terraform init, plan, and apply is crucial because if i run "terraform apply" before "terraform init", i would run into an error because terraform init needs to set up my project first by downloading necessary plugins and setting up the state file, which is a file Terraform uses to track the current state of my infrastructure. There actually aren't any errors if i skip terraform plan. This is because terraform apply will automatically run a plan and ask for confirmation before making any changes. But, skipping the explicit terraform plan means i might miss reviewing these changes in detail. It's best practice to run a plan to catch any potential errors or unexpected results before they affect my live infrastructure.

Michael K.
NextWork Student